

COMI 1150: Relational and Boolean Operators

Assignment Description and Background

In this assignment, you will be continuing to develop your skills working with conditional statements and combine them with the use of various boolean and relational operators.

The problems increase in difficulty as you progress. Each one is designed to introduce or reinforce a **pattern** that you will reuse constantly as a programmer: checking a value against a threshold, testing whether a value falls inside a range, checking whether a value belongs to any of several separate ranges, and matching text no matter how the user capitalizes it. Read each problem carefully and think about the pattern it is teaching, not just the specific answer.

Place each problem in a separate `.py` file.

Note

This assignment is worth **100 points** total. Each problem shows its point value, and every problem ends with a **graded requirements** breakdown that lists how those points are earned. Use that breakdown as a checklist before you submit.

Problem Specifications

Problem 1: Relational Operators and Integers

1. For this problem, your program will ask the user for two integers. It will then output the Boolean result of what the various relational operators produce when used with those values.

8

Write some statements to get the two integers from the user. Store the values in variables named `firstInt` and `secondInt`. Make sure that your prompts are identical to what is shown in the sample console session below.

Warning

Note: At this point in the course, I will no longer remind you to use the correct datatype annotations on your variables. You should know to do this by now...

Your program output should look like this (example user responses are shown in blue):

Sample Output

```
What is the first integer? 13
What is the second integer? 27
```

Your program will then use a series of print statements to display the result of using the following relational operators: `==`, `!=`, `<`, `<=`, `>`, and `>=`.

For example, looking at the above console session we can see that the user typed 13 and 27. If we were to consider the `>` operator, it would display **False** because 13 is not greater than 27.

Write a single statement to insert a blank line into the output (separating the questions from the output).

Write six (6) statements to display output that states the relational expression and their result in sentence form. Be sure that your output matches the formatting shown in the sample console session exactly.

An example of running your program might look like this:

Console Session 1

```
What is the first integer? 13
What is the second integer? 27

13 == 27 is False.
13 != 27 is True.
13 < 27 is True.
13 <= 27 is True.
13 > 27 is False.
13 >= 27 is False.
```

Console Session 2

```
What is the first integer? 14
What is the second integer? 14

14 == 14 is True.
14 != 14 is False.
14 < 14 is False.
14 <= 14 is True.
14 > 14 is False.
14 >= 14 is True.
```

Hint

Hint: Many students try to do this using **if-else** statements and are trying to display the correct True/False value based on the result of evaluating the **if** condition. This is NOT the correct approach here. There should not be any **if** statements in your solution to this problem.

Remember, the result of evaluating a relational expression is a True/False value and those things can be printed directly!

Graded requirements:

- a. Reads two integers into `firstInt` and `secondInt` with the correct data types. 2
- b. Prints the result of all six relational operators directly, with no **if** statements. 4
- c. Output (including the blank line and sentence wording) matches the sample sessions exactly. 2

Problem 2: Below Freezing

2. Water freezes at 32 degrees Fahrenheit. In this problem you will write a program that warns the user when the temperature has dropped below that point.

6

Write a statement that asks the user for the current temperature in Fahrenheit and stores the result as a `float` in a variable named `temp`.

Using an `if-else` statement, your program should decide what to display:

- If the temperature is **below** freezing, warn the user about ice.
- Otherwise, tell the user it is above freezing.

Match the formatting shown below (user responses in `blue`):

```
Enter the current temperature in Fahrenheit: 28.5
Brr! 28.5 degrees is below freezing. Watch for ice!
```

Console Session 1

```
Enter the current temperature in Fahrenheit: 50
50.0 degrees is above freezing.
```

Console Session 2

Hint: “Below freezing” means the temperature is strictly **less than** 32. Think carefully about which relational operator says “less than” versus “less than or equal to.” What message would you want to show if the temperature were **exactly** 32? Testing that boundary value is a great way to confirm you picked the right operator.

Hint

Graded requirements:

- a. Reads the temperature as a `float` into `temp`.
- b. Uses the correct relational operator to test “below freezing” (strictly less than 32).
- c. Displays the correct message for each case, matching the format shown.

2

2

2

Problem 3: Yes or No?

3. Programs constantly need to ask the user a yes-or-no question. The tricky part is that a user might type their answer in different ways: `yes`, `Yes`, or `YES`. Your program should understand all of the common capitalizations.

8

Write a statement that asks the user `"Do you want to continue? (yes/no): "` and stores their response in a variable named `answer`.

Then, using `if`, `elif`, and `else`, your program should respond as follows:

- If the user typed **yes** in any of these forms — `yes`, `Yes`, or `YES` — print a message that you are continuing.
- If the user typed **no** in any of these forms — `no`, `No`, or `NO` — print a message that you are stopping.
- For anything else, print that you did not understand the response.

Console Session 1

Do you want to continue? (yes/no): YES
Great, let's keep going!

Console Session 2

Do you want to continue? (yes/no): no
Okay, stopping here.

Console Session 3

Do you want to continue? (yes/no): Maybe
Sorry, I didn't understand that response.

Warning

Do not use `.upper()`, `.lower()`, or any other string method to solve this. Compare the user's input directly against each accepted spelling. You have not learned those methods yet, and the goal here is to practice combining comparisons with a Boolean operator.

Hint

Hint: A single `if` condition can test several possibilities at once. If you need the condition to be true when the input matches **any one of** several accepted spellings, which Boolean operator combines those comparisons? Remember to include all three capitalizations for each answer.

Graded requirements:

- a. Reads the user's response into `answer`. 1
- b. Uses the correct Boolean operator(s) to accept all three spellings of a **yes** answer. 3
- c. Uses the correct Boolean operator(s) to accept all three spellings of a **no** answer. 3
- d. Handles any other response with the "did not understand" message. 1

Problem 4: Valid Percentage

4. Before a program trusts a number the user typed, it usually needs to check that the number is **reasonable**. A percentage, for example, only makes sense if it is between 0 and 100. 8

Write a statement that asks the user to enter a test score and stores it as an `int` in a variable named `score`.

A score is valid only when it is **at least 0 and at most 100** (both 0 and 100 count as valid). Using an `if-else` statement, print whether the score the user entered is valid.

Console Session 1

```
Enter a test score (0-100): 87
87 is a valid score.
```

Console Session 2

```
Enter a test score (0-100): 145
145 is not a valid percentage.
```

Console Session 3

```
Enter a test score (0-100): -4
-4 is not a valid percentage.
```

Hint

Hint: A number is “in range” only when **two** things are true at the same time: it is not too low, and it is not too high. That means you need two separate comparisons joined by a Boolean operator. Which operator requires **both** comparisons to be true? Be sure to test the exact boundary values (0 and 100) to confirm they are treated as valid.

Graded requirements:

- a. Reads the score as an `int` into `score`. 2
- b. Uses the correct Boolean operator to test the full range, with both endpoints counted as valid. 4
- c. Displays whether the score is valid or not, matching the format shown. 2

Problem 5: Product Recall Check

5. Manufacturers frequently announce a recall that applies only to products whose serial numbers fall within certain **ranges**. A single product line might have several separate ranges affected, with perfectly good units in between. In this problem you will check whether one unit is part of a recall.

10

A company has recalled units whose serial numbers fall within **any** of these ranges (the endpoints are included):

- 10450 through 10999
- 22000 through 22350
- 38500 through 38575

Write a statement that asks the user for a serial number and stores it as an `int` in a variable named `serial`. Then, using an `if-else` statement, report whether that unit is affected by the recall.

Console Session 1

Enter the serial number: 10500
Serial 10500 IS affected by the recall. Please return this unit.

Console Session 2

Enter the serial number: 22350
Serial 22350 IS affected by the recall. Please return this unit.

Console Session 3

Enter the serial number: 11000
Serial 11000 is not affected by the recall.

Hint

Hint: This builds directly on the “within a range” pattern from Problem 4. First figure out how to test whether the serial falls inside **one** range — that took two comparisons joined by a Boolean operator. A unit is affected if it falls in the first range **or** the second **or** the third, so test each range separately and then combine those tests with the operator that is true when **any one** of them is true. Wrapping each individual range test in parentheses will keep your logic readable and correct.

Graded requirements:

- a. Reads the serial number as an `int` into `serial`. 2
- b. Correctly tests each individual recall range, counting the endpoints as affected. 4
- c. Combines the range tests so that a match in **any** range is reported as affected. 2
- d. Displays whether the unit is affected, matching the format shown. 2

Problem 6: Compass Direction

6. This problem combines the capitalization matching from Problem 3 with the multi-way decision structure you have been building. 10

Write a statement that asks the user to enter a direction (N, S, E, or W) and stores it in a variable named `direction`. Your program must accept either uppercase **or** lowercase for each letter.

Using an `if-elif-else` chain, print the full name of the direction. If the user enters anything that is not one of the four directions, print that it is unknown.

Console Session 1

Enter a direction (N/S/E/W): e
Heading East.

Enter a direction (N/S/E/W): S
Heading South.

Enter a direction (N/S/E/W): x
Unknown direction.

Hint

Hint: Each direction can arrive in two forms — for example, the north branch should trigger on both N and n. Inside a single branch's condition, join those two forms with a Boolean operator so that **either** one activates that branch.

Graded requirements:

- a. Reads the direction into `direction`. 2
- b. Uses an `if-elif-else` chain that covers all four directions. 2
- c. Accepts both capitalizations of each letter using the correct Boolean operator. 4
- d. Displays the correct full name, or “Unknown direction” for anything else. 2

Problem 7: Using Boolean Operators

7. In this problem, you will explore working with strings, Booleans, and Boolean operators. 14

Begin by writing statements to ask the user for two Boolean values. The expected responses for these will be the letters “T” or “F” (in any capitalization). Store these values in variables named `stringB1` and `stringB2`.

Hint

Note: Despite the fact that we are asking for Booleans, for this to work correctly, we first need to get the values from the user as Strings, so make sure that you have the correct datatypes and annotations. You would not ordinarily include the datatype as part of the variable name, but I have done so here to make it a bit clearer for you.

The next step is to use conditional statements to evaluate the strings and assign values to some Boolean variables.

In the example code below, we are looking to see if the user typed “T” for the first value, and if so, we are creating a Boolean variable named `a` and assigning it `True`, otherwise `a` will be `False`.



Python

```
if (stringB1 == "T"):
    a:bool = True
else:
    a:bool = False
```

Add the above code to your program. You will do the same thing with `stringB2` and a variable named `b`.

As noted above, your program should handle user responses in any capitalization (including uppercase and lowercase). The code that is provided here does not do that correctly.

Modify the code that checks `stringB1` and `stringB2` so that the expression correctly handles either form of capitalization.

Hint

Reminder: The Boolean operators available to you to use in Python are `and`, `or`, and `not`.

Now that we have the user's two Boolean values stored as Boolean variables, we will acquire one additional piece of information from the user. We will ask the user what they want to do with these two values. They will have a choice of performing an AND operation or an OR operation.

Write a statement that asks the user if they want to AND the values or OR the values. Store their response in a variable named `boolOp`.

Warning

Observation: The use of `boolOp` as a variable name is a tricky one. It is short for "boolean operation". However, students often read this as "boo ten P", which makes no sense at all. What is happening here? Well, lowercase L looks like the number 1 and capital O is often mistaken for the number 0. Keep this in mind when choosing variable names. It can be very easy to misread them!

If the user types "and", then your program will `and` together the values of `a` and `b` and display the result. If they type "or", then your program will `or` together the values of `a` and `b` and display the result.

Write the required statements so that your program will correctly display the result of the chosen operation on the provided values. Your program needs to handle the user typing all uppercase or all lowercase versions of "and" and "or" (no need to handle mixed cases for now).

Example program session (user responses in blue):

Console Session 1

```
Enter the first boolean value (t/f): T
Enter the second boolean value (t/f): F
Do you want to AND or OR the values together? or

The result is: True
```

```
Enter the first boolean value (t/f): f
Enter the second boolean value (t/f): T
Do you want to AND or OR the values together? AND

The result is: False
```

Warning

Note: Pay attention to the examples shown above, and notice that they work for both uppercase and lowercase letters for all inputs. Your program should handle this by using Boolean operators and testing for both uppercase and lowercase versions of the expected inputs.

Hint

Hint: When evaluating the user's request for **and** or **or**, there is no need to test all the combinations of **True/False** values. You can use the Boolean operators directly. Both the **if** and the **else** portions of this code should only require a single statement each.

Alert

Warning: DO NOT use `.upper`, `.lower`, lists, or any other way of comparing different capitalizations other than direct comparisons separated by a Boolean operator for this assignment. I am specifically evaluating your use of Boolean operators here.

Graded requirements:

- a. Reads both values as strings into `stringB1` and `stringB2`. 2
- b. Converts each response to a Boolean (**a** and **b**), correctly handling either capitalization of T/F. 4
- c. Reads the operation choice into `boolOp` and handles the uppercase and lowercase forms of "and"/"or". 3
- d. Uses the Boolean operators directly (not a test of every True/False combination) to compute the result. 3
- e. Displays the correct result, matching the format shown. 2

Problem 8: Account Login

8. A login check must confirm that the user gave **both** the right username **and** the right PIN. If either one is wrong, access is denied. In this problem you will practice requiring two conditions to be true at once.

10

Write statements that:

- Ask for a username and store it in a variable named `username`. Accept either `admin` or `ADMIN`.
- Ask for a PIN and store it as an `int` in a variable named `pin`. The correct PIN is 1234.

Using an **if-else** statement, grant access only when the username is one of the accepted spellings **and** the PIN is correct. Otherwise, deny access.

Console Session 1

```
Enter username: admin
Enter PIN: 1234
Access granted. Welcome, admin!
```

Console Session 2

```
Enter username: ADMIN
Enter PIN: 1234
Access granted. Welcome, ADMIN!
```

Console Session 3

```
Enter username: admin
Enter PIN: 0000
Access denied.
```

Console Session 4

```
Enter username: guest
Enter PIN: 1234
Access denied.
```

Hint

Hint: This condition mixes two different ideas. Accepting **either** spelling of the username is an “or” idea, while requiring **both** the username and the PIN to be correct is an “and” idea. When you combine **and** and **or** in a single expression, wrap the “either spelling” part in parentheses so it is evaluated as one unit before it is combined with the PIN check. Also make sure the type you read for the PIN matches the type you compare it against.

Graded requirements:

- a. Reads the username into `username` and the PIN as an `int` into `pin`. 2
- b. Accepts either capitalization of the username using the correct Boolean operator, grouped with parentheses. 3
- c. Requires **both** the username and the PIN to be correct using the correct Boolean operator. 3
- d. Displays “Access granted” (with the name) or “Access denied” to match each case. 2

Problem 9: Movie Ticket Pricing

9. A movie theater charges different prices depending on the customer's age, and it offers a discount on Tuesdays. In this problem you will combine a range-based decision with a follow-up condition.

12

Write a statement that asks for the customer's age and stores it as an `int` in a variable named `age`. Determine the **base** ticket price from these non-overlapping ranges:

- Under 13 (a child): \$8
- 13 through 64 (an adult): \$12
- 65 and older (a senior): \$9

Then ask "Is today Tuesday? (yes/no): " and store the response in a variable named `today`. If the answer is **yes** (in any capitalization: **yes**, **Yes**, or **YES**), subtract \$2 from the ticket price. Finally, print the ticket price the customer will pay.

Console Session 1

```
Enter your age: 10
Is today Tuesday? (yes/no): no
Your ticket price is $8.
```

Console Session 2

```
Enter your age: 70
Is today Tuesday? (yes/no): YES
Your ticket price is $7.
```

Console Session 3

```
Enter your age: 30
Is today Tuesday? (yes/no): no
Your ticket price is $12.
```

Hint

Hint: Break this into two separate decisions instead of trying to solve it all at once. First, use an `if-elif-else` chain to figure out the **base** price from the age brackets and store it in a variable. Then, as a completely separate step, check whether it is Tuesday and adjust that price variable if the discount applies. Keeping the two decisions apart keeps each one simple.

Graded requirements:

- a. Reads the age as an `int` into `age`. 2
- b. Determines the correct base price from the three age brackets. 4
- c. Reads the Tuesday response and applies the \$2 discount only when it means yes (any capitalization). 4
- d. Displays the final ticket price, matching the format shown. 2

Problem 10: Ride Eligibility

10. This final problem is a **capstone**: it brings together a threshold check, a range check, Boolean operators, a nested decision, and capitalization matching — all of the patterns you practiced earlier in this assignment.

14

An amusement park ride has the following rules:

- You must be at least 48 inches tall to ride at all.
- If you are tall enough **and** at least 8 years old, you may ride by yourself.
- If you are tall enough but **under** 8 years old, you may ride only if you are accompanied by an adult.
- If you are under 48 inches, you may not ride.

Write statements that ask for the rider's height in inches (store as a **float** named **height**) and age (store as an **int** named **age**). Then use your decision structures to print the correct message. When the rider is tall enough but under 8, ask an additional question — "Are you with an adult? (yes/no): " — and decide based on the response (accept **yes**, **Yes**, or **YES**).

Console Session 1

```
Enter your height in inches: 52
Enter your age: 10
You may ride!
```

Console Session 2

```
Enter your height in inches: 52
Enter your age: 6
Are you with an adult? (yes/no): yes
You may ride with an adult.
```

Console Session 3

```
Enter your height in inches: 52
Enter your age: 6
Are you with an adult? (yes/no): no
Sorry, you may not ride yet.
```

Console Session 4

```
Enter your height in inches: 40
Enter your age: 10
Sorry, you are not tall enough to ride.
```

Hint: Notice that the very first thing that matters is the height requirement — nothing else can happen until that is satisfied. That suggests checking the height **first** with an outer **if**. Only **inside** the “tall enough” branch do you need to worry about age, which leads to a **nested** decision. Look at the console sessions carefully: they tell you exactly which message goes with each combination of height, age, and adult response. Trace each session through your logic before you consider yourself finished.

Graded requirements:

- a. Reads `height` as a `float` and `age` as an `int`. 2
- b. Uses an outer decision to enforce the 48-inch height requirement first. 3
- c. Uses a nested decision on age to separate “ride alone” from “needs an adult”. 4
- d. Asks the adult question and interprets it correctly (any capitalization of yes). 3
- e. Displays the correct message for every case, matching the console sessions. 2

Testing Your Program

A program that runs without crashing is not the same as a program that is **correct**. You need to deliberately choose inputs that prove each decision works. Using Python comments in your code, answer the following questions:

1. For Problem 1, how do you know the program is displaying the correct results for these comparisons? What pair of values would let you confirm each of the six operators individually?
2. For the range problems (Problems 4, 5, and 9), the most common mistakes happen right at the **boundaries** between ranges. What values would confirm that a score of exactly 0 and exactly 100 are valid, that a serial number at the very edge of a recall range (and one just outside it) is classified correctly, and that ages of exactly 13 and exactly 65 land in the correct ticket bracket?
3. For the capitalization-matching problems (Problems 3, 6, and 7), how can you be certain you handle every accepted spelling? Is there a reasonable input a real user might type that your program would incorrectly reject? What is it?
4. For Problem 10, list every distinct path through your logic (for example: tall enough and old enough; tall enough but young and with an adult; and so on). What set of test inputs exercises every one of those paths at least once?

Documentation

Program Header

Your program will contain the following in comments at the top of your file:

- Your name
- Course Title and Section
- Date Completed

Statements

For this assignment, every line of code will contain a comment above the code that completely describes what the line of code is meant to do.

If your code does not successfully perform the task, describe what you think is happening and also describe what you tried to do to correct it.

Checklist

- Comments
- Good variable name choices
- Program output matches what is shown in the assignment
- Program has no syntax errors and runs without crashing
- Program provides the correct calculations for a given set of inputs
- Any required questions were answered in your comments

Introduction

In addition to skills and knowledge, The Community College of Rhode Island aims to teach students appropriate Ethical and Professional Standards of Conduct. The Academic Honesty Policy exists to inform students and Faculty of their obligations in upholding the highest standards of professional and ethical integrity. All student work is subject to the Academic Honesty Policy. Professional and Academic practice provides guidance about how to properly cite, reference, and attribute the intellectual property of others. Any attempt to deceive a faculty member or to help another student to do so will be considered a violation of this standard.

Instructor's Intended Purpose

The student's work must match the instructor's intended purpose for an assignment. While the instructor will establish the intent of an assignment, each student must clarify outstanding questions of that intent for a given assignment.

Unauthorized or Excessive Assistance

The student may not give or get any unauthorized or excessive assistance in the preparation of any work. This includes using tutoring or related services to complete homework assignments or exams as those submissions must be your own original work.

Declaration of Authorship

Online submissions of an assignment to a student's account or placing your name on any course deliverable is a statement that the student is the author of that work and that the student has not received or given inappropriate assistance in completing it and that the student has complied with the Academic Honesty Policy in that work. Any referenced work must be clearly documented, cited, and attributed, regardless of media or distribution formats. Note that finding solutions (in whole or in part) to any assignment online (or acquiring them from another student or tutor) and using them as your own, regardless of whether or not changes have been made, constitutes plagiarism.

Consequences

Instances of academic dishonesty are handled on a case-by-case basis, and the consequences may vary depending on the severity of the offense. Possible sanctions include, *but are not limited to*, the following: (1) Require that a makeup test is taken, or a makeup paper is written; (2) Reduce the grade on the paper or exam; (3) Reduce the final grade in the course by one or more levels; (4) Assign a grade of F to the exam or paper; (5) Assign a grade of F for the course. A student may appeal these decisions according to the Academic Grievance Procedure documented in the Student Handbook.

www.ccri.edu/advising/new_students/student_handbook/conduct

Academic Dishonesty (Continued)

By enrolling and participating in this course, you acknowledge that you have read and agree with the college Academic Dishonesty Policy and specifically the items listed below.

You are allowed to:

- ✓ Use any of the materials provided to you as part of this course, including content from your textbook, PowerPoint slides, videos, and lecture notes.
- ✓ Consult online and other sources for help with general course topics that are not specific to the assignment (for example, a YouTube video about how a **while** loop works or about how to use a specific HTML element).
- ✓ Ask the professor questions about the course content or the assignment.
- ✓ Other than the above, submitted work will be the result of your own independent thinking and efforts.

You are NOT allowed to:

- ✗ Search online for solutions to any part of the assignment (Chegg, Bartleby, etc.) or use any unauthorized assistance.
- ✗ Discuss or get help on the assignment in any online system (Facebook, YouTube, Discord, programming websites/forums, ChatGPT or other AI systems, etc.)
- ✗ Work together with others (students or other) on the assignment.
- ✗ Copy code from others (other students, online sources, etc.)
- ✗ Present the work of others (in whole or in part) as your own.
- ✗ Fabricate data or misrepresent the origins of any data used as part of your assignment.
- ✗ Help others to violate any portion of the academic dishonesty policy.

Warning

If you are unsure if something is allowed or not, be sure to email me to confirm. "*I didn't know*" will not be accepted as a reason for violating this policy.

Alert

The penalty for violating the Academic Dishonesty Policy is a failing grade for the semester and being reported to the Dean for disciplinary action. In the case of students who are found to be sharing assignment/exam problems/solutions online, I will push for expulsion from the college.

Assignment Submission

You will submit your work into Blackboard on or before the due date/time. For courses that meet in-person, if the specified time is “by the start of class,” it means that the work should be submitted prior to that time. For assignments that are due “at the end of the day,” it means that they should be submitted prior to 11:59 PM EST on the day specified.

Acceptable File Types

In this course, you will be submitting .zip files containing *all* required files to build your solution, including preserving any required folder structures.

Alert

DO NOT submit .pdf, .docx, .jpg, .txt or anything other than your source files unless specifically asked.

You **MUST** place your code in a ZIP. DO NOT submit 7Z, RAR or other container files.

Warning

Note for Python Coders: The only thing that should be present in your .py file is actual Python code. I should not see any content from the console, and it should not be the result of an interactive session. If your submission has lots of >>> characters in it, you have done something wrong...

Incorrect Files

Assignments may allow for multiple submission attempts. I will grade the most recent on-time submission. It is your responsibility to ensure that you are submitting the correct file(s). I will not accept assignment submissions by email. If you continually submit the wrong files your work will be rejected without opportunity for resubmission.

Successful Compilation and Execution

All submissions should be able to be executed and should not crash due to syntax errors.

Blackboard Comments

No work that is part of your assignment submission should go in the Comments section in Blackboard. If you type something in the Comments box, note that I will not see it until I grade the submission.

Grading Criteria

	Unacceptable < 60	Acceptable ≥ 60	Good ≥ 80	Excellent – 100
Specifications and Correctness	Program does not function at all or only under very limited circumstances.	Most problems attempted and program mostly works but does not meet some specifications.	All problems attempted and mostly works with minor problems. Possible violation of specifications.	Meets all specifications and requirements.
Code Quality and Readability	Multiple major issues with code design or program organization. Poor identifier names or whitespace usage. Program may or may not run correctly.	Major issues with code design or program organization. Identifier names or whitespace usage need improvement. Program may or may not run correctly.	No errors. Minor issues with code design, identifier names, or program organization.	No errors. Code is clean and well-organized. Control structures well chosen.
Documentation	Contains no meaningful comments.	May contain some comments, but several items remain undocumented, or the comments are unclear.	May have some areas that require additional documentation or is <i>overly</i> commented.	Program includes a sufficient number of meaningful comments.
Execution	Does not run or runs with severe errors. May not perform the requested task at all. Poor choices of program implementation leading to inefficient design.	Runs without errors in most circumstances. Mostly good coding choices, but numerous opportunities to improve the efficiency of the program.	Runs without errors in most circumstances. Efficient coding choices, possibly with room for improvement.	Runs without errors and performs all required tasks. Efficient coding choices.

Additional Notes: Some grading criteria may not apply to all assignments, and some criteria will be assessed more strictly as the course progresses. If a program does not meet the specifications at all or is entirely incorrect, no credit will be received for any other criteria. Other details in this document may take precedence over anything on this page. Check your grading feedback for specific details.

Submitted programs should be free of any (obvious) syntax errors. If during the normal operation of your program, a syntax error occurs that results in the termination of your program, it **MUST** be corrected prior to submission. Even if a program does not fully solve the problems in this assignment, there should be no syntax errors. Any academic dishonesty violation will result in, at a minimum, the loss of all points.